

CLASE 1

2024 - PROGRAMACION CONCURRENTE

FAC.DE INGENIERIA - UNJu

Paradigma Orientado a Objetos

Prof. Adj. Esp.Ing. José Farfán

JTP: Mg. Nélide Cáceres – Dr.Federico Medrano

Ay: Lic. Felipe Mullicundo

Programación Orientada a Objeto (POO)





PARADIGMA ORIENTADO A OBJETOS

RAZONES DE SU APARICIÓN

Falta de Portabilidad del Código y Reusabilidad

Código difícil de modificar

Ciclos de Desarrollo largos

Técnicas de Codificación NO Intuitivas

CARACTERÍSTICAS BÁSICAS

Basado en Objetos

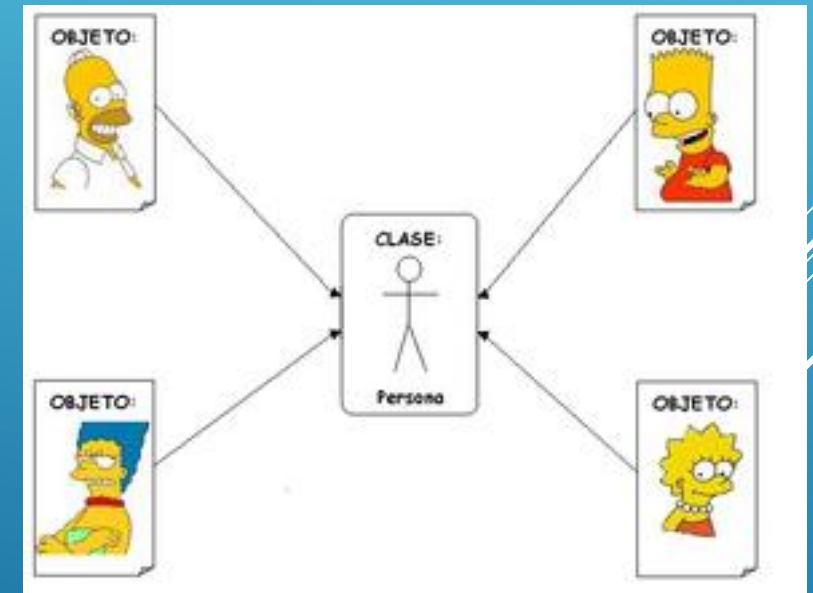
Basado en Clases

Capaz de tener HERENCIA de clases



PARADIGMA ORIENTADO A OBJETOS

- Def. POO: Metodología de desarrollo de aplicaciones en la cual éstas se organizan como **colecciones cooperativas de objetos**, cada uno de los cuales representan una **instancia de alguna clase**, y cuyas clases son miembros de **jerarquías de clases** unidas mediante relaciones de **herencia**. (Grady Booch)



- Def. Objeto: conjunto complejo de datos y programas que poseen estructura y forman parte de una organización. (Cruz del Valle)



PARADIGMA ORIENTADO A OBJETOS

Estructura de un Objeto

Partes de un OBJETO

Relaciones

Permiten que el objeto se inserte en la **organizac.** y están formadas x punteros a otros objetos

Propiedades

Todo objeto puede tener cierto nro de prop., c/u de las cuales tendrá, a su vez, 1 o varios valores (matrices, vectores, listas, etc.). Además, los valores pueden ser de cualquier tipo (numérico, alfabético, etc.) .

Métodos

Son las operaciones q pueden realizarse sobre el **objeto**, normalmente están incorporados en forma de programas (código) q el objeto es capaz de ejecutar y q también pone a disposición de sus descendientes a través de la herencia

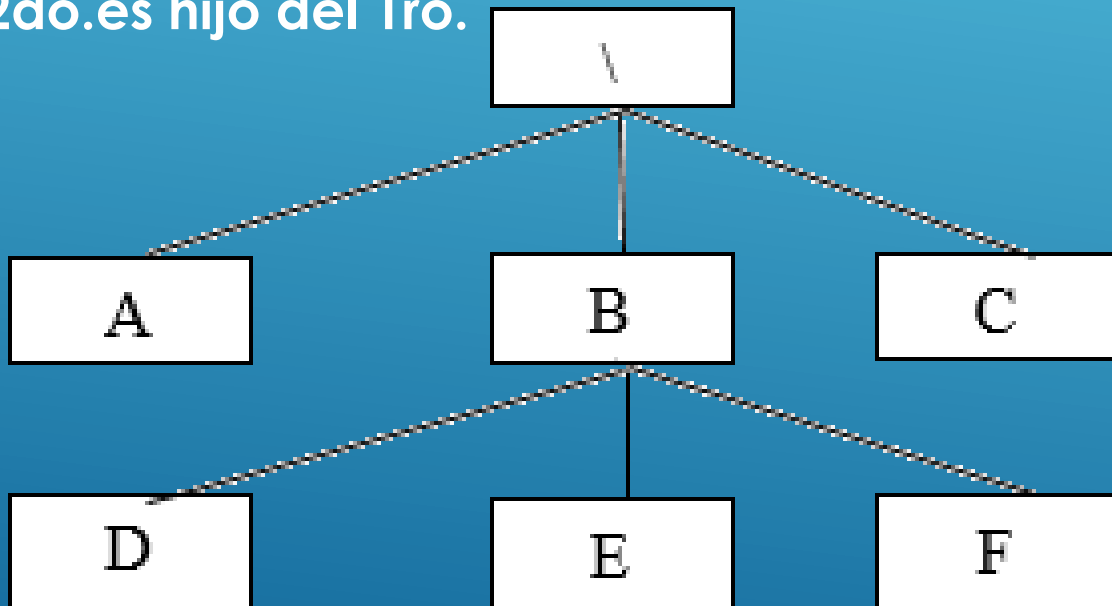


PARADIGMA ORIENTADO A OBJETOS

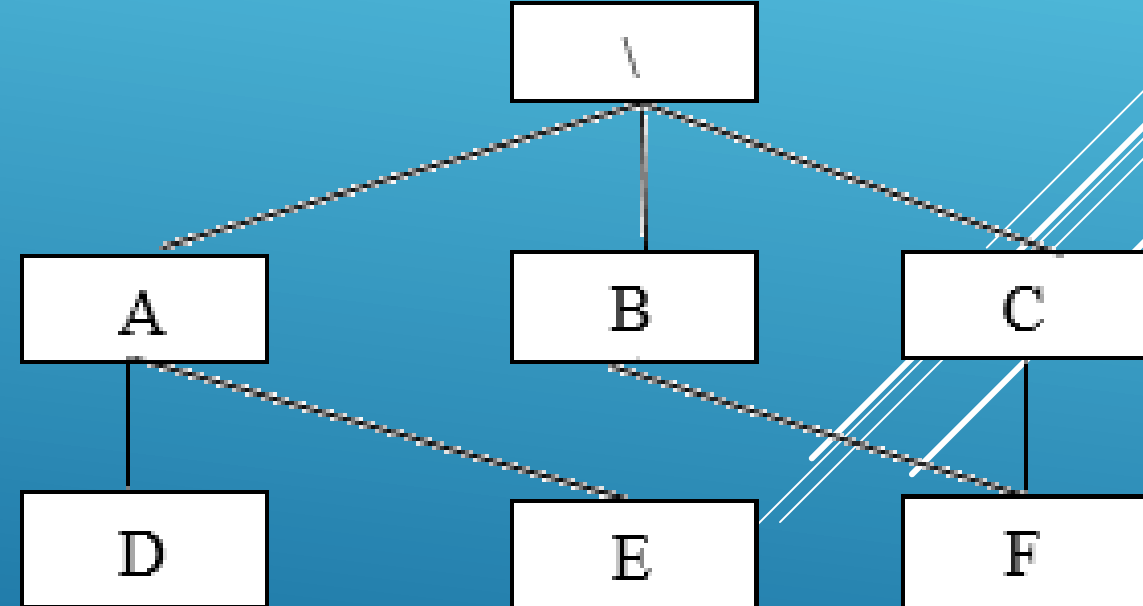
RELACIONES: tipos

RELACIONES JERARQUICAS

Son esenciales p/la existencia misma de la aplicación xq la construyen. Son bidireccionales, es decir, 1 objeto es padre de otro cuando el 1er.objeto se encuentra situado inmediatamente encima del 2do.en la organiz.en la q ambos forman parte. Si un objeto es padre de otro, el 2do.es hijo del 1ro.



Org.Jerárq.Simple → 1 hijo tiene 1 solo padre



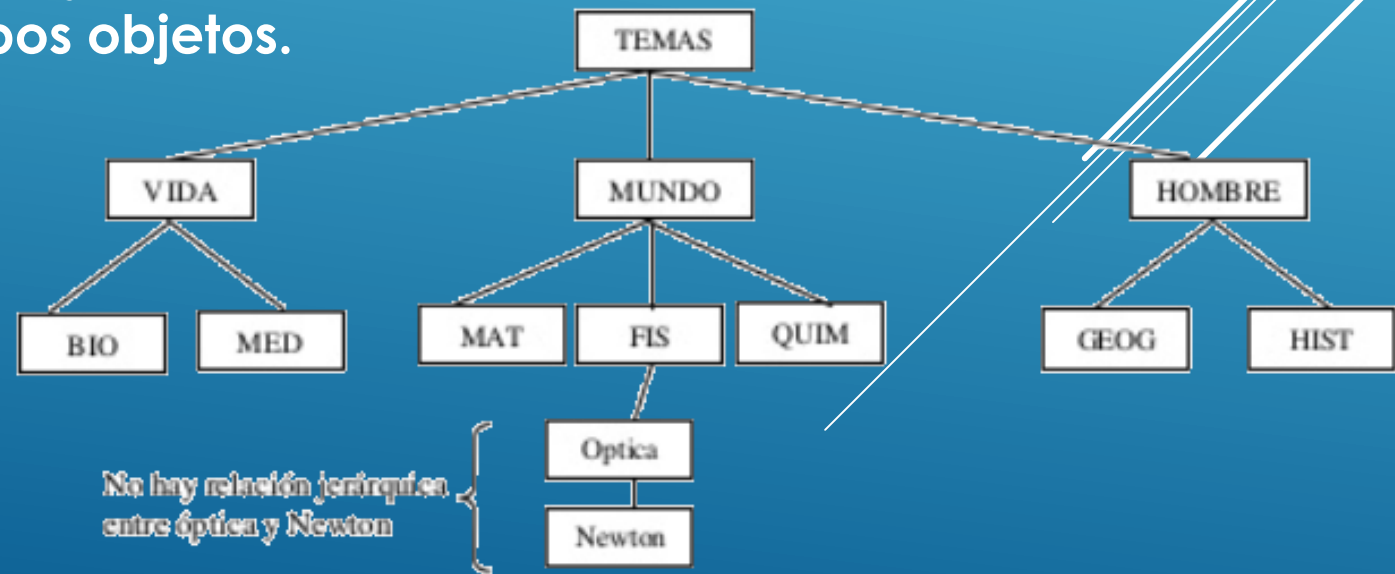
Org.Jerárq.Compleja → 1 hijo tiene varios padres

PARADIGMA ORIENTADO A OBJETOS

RELACIONES: tipos (cont.)

RELACIONES SEMANTICAS

NO tienen q ver con la organiz.de la q forman parte los objetos q las establecen. **Sus propied.** solo dependen de los objetos en sí mismos (de su significado) y no de su posición en la organiz. Ej.: diccionario informatizado q permita al us.obtener la definición de 1 palabra. Las palabras son objetos y la organiz.jerárq.es la q proviene de forma natural de la estruct.del conocim. sobre el mundo. La raíz se llama TEMAS. De éste término genérico descienden 3 grandes ramas de objetos llamadas VIDA, MUNDO y HOMBRE. Vida comprende las Cs biológicas: Biología y Medicina. De mundo las Cs de la naturaleza inerte: Matemática, Física, Química y Geología. Hombre comprende las Cs humanas: Geografía, Historia, etc. **Se establece la relación trabajó entre los objetos NEWTON y OPTICA, la cual significa que Newton trabajó en óptica, esta relac.es semánt.,** pues no hay connotación jerárq.entre NEWTON y OPTICA y su interpret.depende del significado de ambos objetos.





PARADIGMA ORIENTADO A OBJETOS

CLASIFICACION DE LAS RELACIONES

Relación De-La-Especie

Se usa al nivel de clase para describir las relaciones entre dos clases similares.

Ej.: programa para dibujar. Permite el dibujo de variados objetos tales como puntos, rectángulos, triángulos y muchos más. Por c/objeto, se provee una definición de clase.

La clase Point define 1pto.x sus coordenadas:

```
class Point {  
  attributes:  
    int x, y  
  methods:  
    setX(int newX)  
    getX()  
    setY(int newY)  
    getY()  
}
```

Un círculo define un punto central y un radio:

```
class Circle {  
  attributes:  
    int x, y,  
      radius  
  methods:  
    setX(int newX)  
    getX()  
    setY(int newY)  
    getY()  
    setRadius(newRadius)  
    getRadius()  
}
```

PARADIGMA ORIENTADO A OBJETOS



CLASIFICACION DE LAS RELACIONES Relación De-La-Especie (cont.)

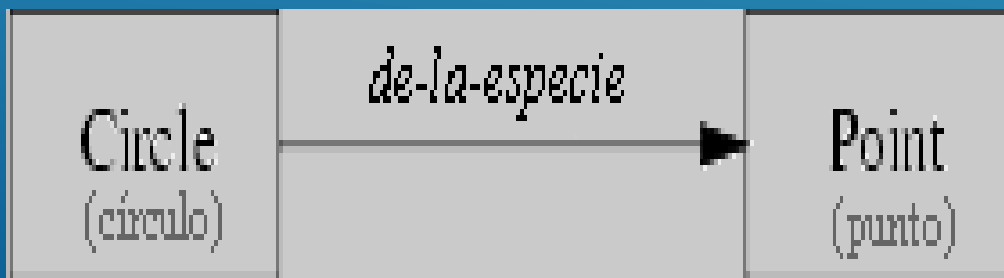
Comparando ambas definiciones de clase:

- **Ambas clases tienen 2 elementos de datos x e y.** En la clase Point estos elementos describen la posición del pto, en el caso de la clase Circle describen el centro del círculo. Así, x e y tienen el mismo significado en ambas clases: **Describen la posición de su objeto asociado x medio de la definic.de 1 pto.**

- **Ambas clases ofrecen el mismo conj.de mét.p/obtener y definir el valor de los 2 elem. x e y.**

- **La clase Circle "añade" un nuevo elem.radius y sus correspondientes métodos de acceso.**

Con las prop.de la clase Point se describe 1 círculo como 1 pto. + 1 radio + mét.p/accederlo.



Las clases se dibujan usando rectángulos. Su nombre empieza con 1 letra mayúscula. Las flechas indican la **dirección de la relación**, de ahí q se deba leer como "Circle es de-la-especie Point".



PARADIGMA ORIENTADO A OBJETOS

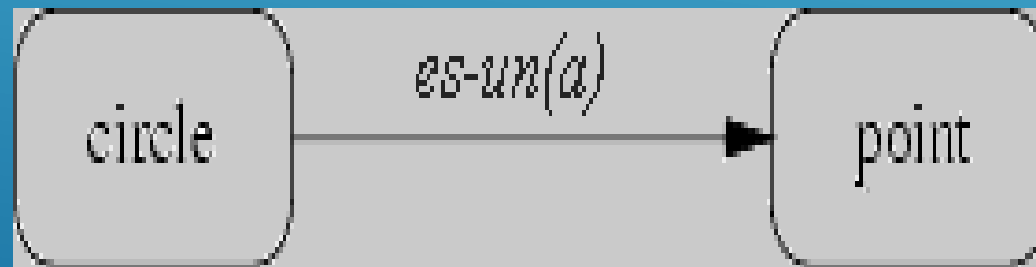
CLASIFICACION DE LAS RELACIONES

Relación Es-Un(a)

Cuando se crean objetos de clases de la relación "De-La-Especie", la relac. se llama "es-un(a)".

La clase Circle es de la especie de la clase Point, 1 instancia de Circle como acircle, es 1 point. C/círculo se comporta como 1pto., se puede mover pto. en la dirección **x** al alterar el valor de **x**.

Similarmente, se mueven círculos en ésta dirección al alterar su valor de **x**.



P/objetos se usan rectáng. con esquinas redondeadas. Su nombre consta de letras minúsculas.

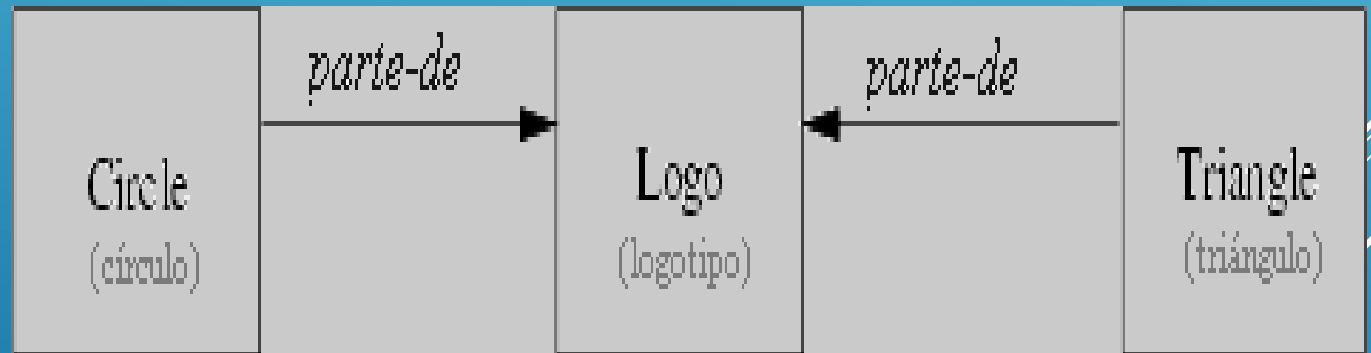
PARADIGMA ORIENTADO A OBJETOS

CLASIFICACION DE LAS RELACIONES Relación Parte-De

A veces se necesita construir objetos haciendo 1 combinación de otros. Esto se hace x la programac. procedim., donde se tiene la estructura o registro p/juntar variados tipos de datos.

En el programa de dibujo se quiere tener 1 figura especial q representa 1 logotipo propio q consiste en 1 círculo y 1 triángulo. Se asume q ya se tiene definida un clase Triangle. El logo consta en 2 partes o, el círculo y el triángulo son parte-de logotipo:

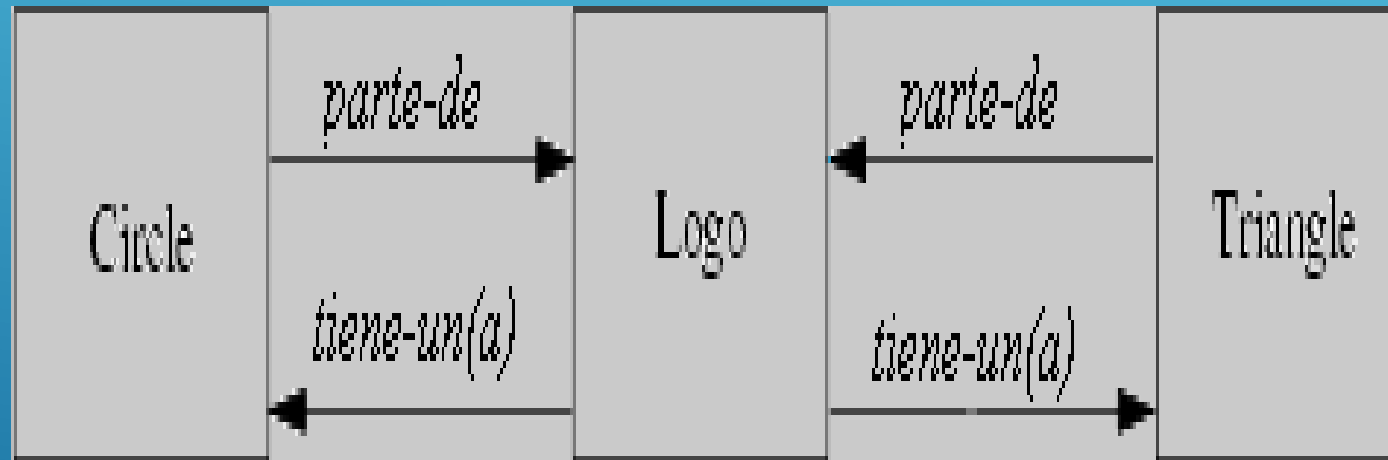
```
class Logo {
  attributes:
    Circle circle
    Triangle triangle
  methods:
    set(Point where)
}
```



PARADIGMA ORIENTADO A OBJETOS

CLASIFICACION DE LAS RELACIONES Relación Tiene-Un(a)

Esta relación **es la inversa de la relación parte-de**. Por lo tanto, se puede añadir esta relación a la ilustración parte-de **añadiendo flechas en la otra dirección**





PARADIGMA ORIENTADO A OBJETOS

PROPIEDADES

Se diferencian con las "variables" xq se pueden heredar de unos objetos a otros. Pueden ser:



PROPIEDADES PROPIAS

Están formadas dentro de la cápsula del objeto.



PROPIEDADES HEREDADAS

Están definidas en 1 objeto diferente, antepasado de éste (padre, "abuelo", etc.).

A veces se las llaman propiedad miembro xq el objeto las posee x el solo hecho de ser miembro de 1 clase.

Las prop. distinguen 1 objeto de los restantes q forman parte de la misma organiz. y tiene valores q dependen de la prop. q se trate. Las prop. de 1 objeto pueden ser heredadas a sus descend. en la organiz.



MÉTODOS

PARADIGMA ORIENTADO A OBJETOS

Son las operaciones q realizan acceso a los datos. Se pueden definir como 1 programa proced. escrito en cualquier leng., q está asociado a 1 objeto determinado y cuya ejecución sólo puede desencadenarse a través de 1 mensaje recibido por éste o sus descendientes.

En el Paradig.Proced.se los conoce como programas, proced., función, rutina, etc., en OOP se usa el término 'método', x la forma de invocarlo, a través de 1 msj y a su campo de acción, limitado a un objeto y a sus descendientes.

Pueden heredarse, x lo tanto se clasifican en:



Métodos propios
Están incluidos dentro de la cápsula del objeto



Métodos heredados
Están definidos en 1 objeto diferente, antepasado de éste (padre, abuelo, etc.).
A veces estos métodos se los llaman métodos miembros xq el objeto los posee x el solo hecho de ser miembro de 1 clase



OBJETOS Características PARADIGMA ORIENTADO A OBJETOS

- ❑ Se agrupan en grupos denominados **clases**
- ❑ Contienen datos internos que definen su estado actual
 - ❑ Soportan **ocultamiento de datos**
- ❑ Pueden **heredar propiedades** de otros objetos
- ❑ Pueden comunicarse con otros objetos enviando o pasando mensajes
 - ❑ Tienen **métodos** que definen su comportamiento

Es 1 entidad lógica q contiene datos y un código especial q indica manipular los datos.

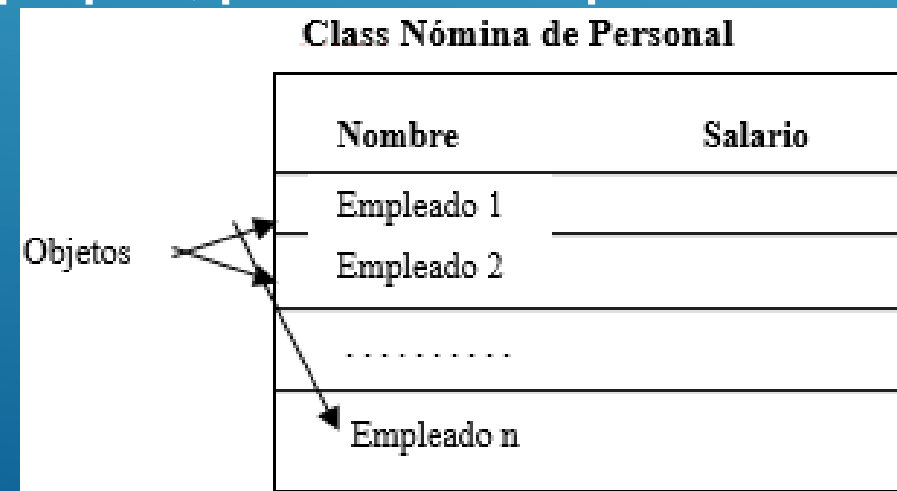
En el momento de la ejecución se puede degradar el diseño del programa.

Son construcciones de program.q se obtienen a partir de entidad. llamadas **clases**. El programador tiene la responsab. de crear clases propias, pero también puede tener acceso a

clases desarrolladas x otros

Ejemplo: diseño de un Objeto.

```
class nomina {nomina empleado;
char nombre[30];
float salario;
}; (nomina es una clase )
    (empleado es un objeto)
```





PARADIGMA ORIENTADO A OBJETOS

CLASE Concepto de una Clase

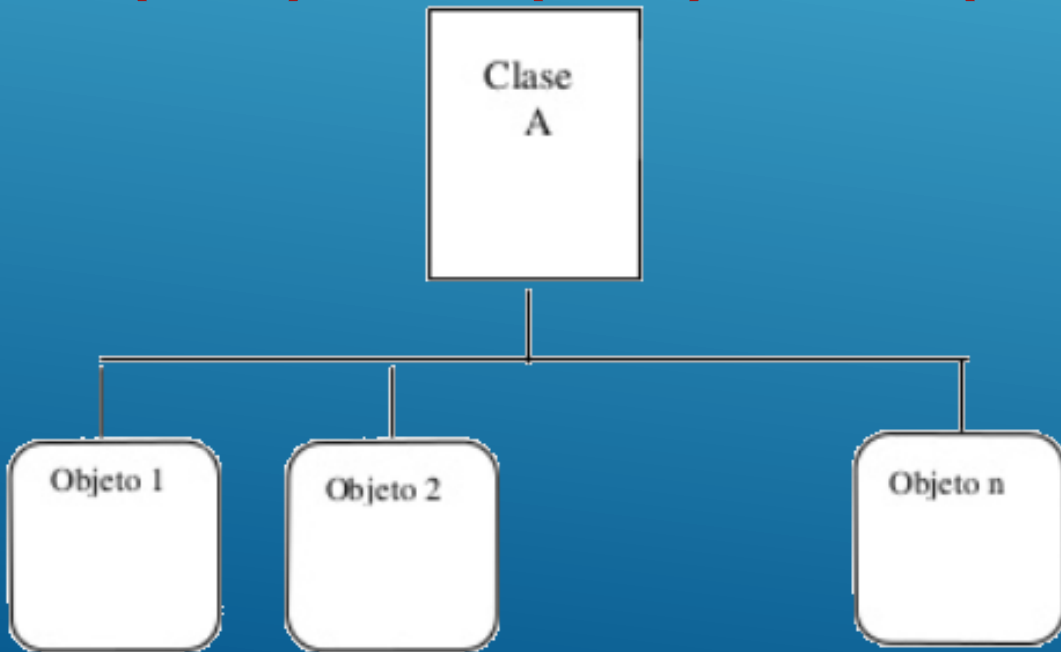
C/objeto es 1ejemplar de 1clase a la q pertenece. Todos los ejemplares de la misma clase tienen el mismo comportamiento (invocan al mismo método) como respuesta a una solicitud.

1clase es 1tipo especial de datos orientado a la creación de objetos q consta de miembros. 1clase es 1tipo de dato que contiene 1 o + elementos llamados datos miembros, y cero, una o más fciones q manipulan esos datos llamados fción miembro. 1clase se puede definir con 1 **estruct.(struct), 1unión (unión) o 1clase (class).**

La sintaxis de una clase es:

```
class nombre_clase
```

```
{  
    miembro_1;    //lista de datos miembros  
    miembro_2  
    miembro_3  
    funcion_miembro_1(); //funciones  
    miembro conocidas  
    funcion_miembro_2 (); //funciones  
    como métodos  
}
```





CLASE

PARADIGMA ORIENTADO A OBJETOS

Identificadores de Diseño de una Clase

P/usar una clase, 1ro.hay q declararla. La declaración de una clase puede aparecer sólo 1 vez en un programa. Ej. declaración de una clase simple:

```
class counter {  
    long count; // Variable miembro de la clase  
public:  
    void SetCount(long);  
    long GetValue( ); };
```

- La palabra clave **class** introduce 1 declaración de clase.
- Después aparece el nombre de la clase.
- Las clases contienen no sólo declaraciones de variables, sino también definic.de fciones.
- Las fciones.contenidas en clases pueden ser tan largas y complejas como uno desee
- Las variables declaradas en 1clase pertenecen a esa clase. Las variables pueden compartirse entre las diferentes instancias de una clase.
- Los identificadores de variables y fciones contenidos en 1clase no coinciden con los identificadores q se usan en otras. 1clase es un mundo con identificadores propios únicos.

PARADIGMA ORIENTADO A OBJETOS

CLASE

Cuerpo de una Clase

En el siguiente ej. la variable count se define dentro del cuerpo de la clase. Por lo tanto, count recibe el nombre de variable miembro de la clase

```
class counter {
    long count;
public:
    count = 3; //se genera un error ya q count no se encuentra definido.
```

Cualquier variable definida en 1 clase tiene 1 campo de acción. Se produce 1 error al intentar el acceso a 1 variable miembro después de la declaración de la clase o fuera del campo de acción de la clase.

CLASE Uso de una Clase

Counter people

Para el objeto people, esta es la forma en se podría usar:

Void main ()

```
{
    counter people;
    //inicializar el objeto people.
    SetValue(0);
    // verificar que se borre
    long value = people.GetValue( );
}
```

CLASE

PARADIGMA ORIENTADO A OBJETOS



Componentes de una Clase

P/definir 1clase se debe tomar en cta.q consta de 2partes: 1declaración y 1implementación.

. La declaración lista los miembros de la clase.

. La implementación o cuerpo define las funciones de la clase.

```
class nomina {nomina empleado;  
char nombre[30];  
float salario;  
}; (nomina es una clase )  
      (empleado es un objeto)
```

Declaración de una clase

```
.....  
class contador{  
    long cuenta;  
public:  
    void leervvalor(long);  
    long obtenervalor( );  
};
```

Fciones miembro de la clase

```
Implementación de una clase  
void contador::leerValor(long valor)  
{  
    cuenta = valor;  
}  
long contador::obtenerValor( )  
{  
    return cuenta;}  
}
```

PARADIGMA ORIENTADO A OBJETOS

ENCAPSULAMIENTO Y OCULTACIÓN

Encapsulamiento

Prop. x la q c/objeto es 1estruct.compleja en cuyo interior hay datos y programas relacionados entre sí, encerrados en 1cápsula (Caract. Fundam.en la OOP).

Ocultación de la Información

Prop. x la q los objetos son inaccesibles, e impiden que otros objetos, us., o incluso los programadores conozcan cómo está distribuida la inform. o q inform. hay disponible. No significa, q sea imposible conocer lo necesario a 1 objeto y a lo q contiene.

Las peticiones de inform.a 1objeto, deben realizarse a través de msjes dirigidos a él, con la orden de realizar la operación pertinente. La respuesta a estas órdenes será la información requerida, siempre que el objeto considere q quien envía el msje está autorizado p/obtenerla. El hecho de q c/objeto sea 1cápsula facilita q 1 objeto determinado pueda ser transportado a otro punto de la organiz., o incluso a otra organización totalmente diferente. Si el objeto se construyó correctamente, sus métodos seguirán funcionando en el nuevo entorno sin problemas. Esta cualidad hace que la OOP sea muy apta para la reutilización de programas.



PARADIGMA ORIENTADO A OBJETOS

ORGANIZACIÓN JERÁRQUICA DE OBJETOS

Los objetos forman siempre 1 organiz.jerárquica, ya q ciertos objetos son superiores a otros.

Existen varios tipos de jerarquías: **SIMPLES** cuando su estructura pueda ser representada x medio de un "árbol". En otros casos puede ser + **COMPLEJOS**. En cualquier caso existen 3 niveles:

- **Raíz de la jerarquía:** Objeto único y especial, se caracteriza x estar en el nivel + alto de la estruct. y se lo llama objeto madre, Raíz o Entidad.
- **Objetos intermedios:** Descienden directamente de la raíz y tienen descendientes. Representan conj.o clases de objetos, q pueden ser muy grales o muy especializados. Reciben nombres genéricos q denotan al conj.de objetos q representan, x ej., VENTANA, CUENTA, FICHERO. En 1 conj.reciben el nombre de clases o tipos si descienden de otra clase o subclase.
- **Objetos terminales:** Son aquellos q descienden de 1 clase o subclase y no tienen descendientes. Suelen llamarse casos particulares, instancias o ítems xq representan los elementos del conj. representado x la clase o subclase a la q pertenecen.

PARADIGMA ORIENTADO A OBJETOS

POLIMORFISMO

El **polimorfismo** es la capacidad de un método de comportarse de manera diferente según la clase del objeto que lo invoque. Esto permite que un mismo mensaje pueda ser enviado a objetos de diferentes clases dentro de una jerarquía de herencia, y que estos objetos respondan de maneras distintas según su implementación.

Por ejemplo, un mensaje **+** enviado a un objeto de tipo **ENTERO** significaría **suma**, mientras que enviado a un objeto de tipo **STRING** significaría **concatenación** (unir dos cadenas de texto).".

La **sobrecarga de un operador** es una característica de algunos lenguajes de OOP que permite redefinir o extender el comportamiento de un operador para trabajar con tipos de datos definidos por el usuario.

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # Sobrecarga del operador +
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

# Creación de dos vectores
v1 = Vector(2, 3)
v2 = Vector(5, 7)

# Suma de los dos vectores usando el operador +
v3 = v1 + v2

print(v3) # Output: (7, 10)
```

PARADIGMA ORIENTADO A OBJETOS

DEMONIOS

Es un tipo especial de métodos, poco frecuente en OOP, q se activa automáticamente cuando sucede algo especial. Es decir, es un programa, como los métodos ordinarios, pero se diferencia de estos xq su ejecución no se activa con un msje, sino q se desencadena automáticamente cuando ocurre un suceso determinado: la asignación de 1 valor a 1 prop. de 1 objeto, la lectura de 1 valor determinado, etc.

Los demonios, cuando existen, se diferencian de otros métodos xq no son heredables y xq a veces están ligados a 1 de las propiedades de 1 objeto, mas q al objeto entero.



PARADIGMA ORIENTADO A OBJETOS

HERENCIA



Hace uso de las relaciones de-la-especie y es-un(a). Las clases q son de-la-especie de otra clase comparten las prop.de esta última. En el ej. visto se puede definir 1 círculo q hereda de 1 pto

```
class Circle inherits from Point
{
  attributes:
    int radius

  methods:
    setRadius(int newRadius)
    getRadius()
}
```

La clase Circle hereda todos los elementos de datos y mét.de la clase Point. No hay necesidad de definirlos 2 veces: Solo se usa los ya existentes, datos y definiciones de mét.

A nivel de objeto se define 1 círculo como 1 pto., debido a q un círculo es-un(a) punto. X ej. se puede definir un objeto círculo (acircle) y establecer sus coordenadas del punto central:

```
Circle acircle
acircle.setX(1)      /* Heredado de Point */
acircle.setY(2)
acircle.setRadius(3) /* Añadido por Circle */
```

PARADIGMA ORIENTADO A OBJETOS

HERENCIA (cont.)

"Es-un(a)" también implica q s puede usar 1 círculo en cualquier circunstancia donde se pueda usar 1pto. x ej., se puede escribir 1 método, move(), el cuál mueve 1pto.en la dirección x:

```
move(Point apoint, int deltax) {
    apoint.setX(apoint.getX() + deltax)
}
```

Ya q círculo hereda de pto., se puede usar esta fcción con 1 argumento círculo p/mover su punto central y, a partir de ahí, todo el círculo :

```
Circle acircle
...
move(acircle, 10) /* Mover el círculo al mover */
/* su punto central */
```

HERENCIA (cont.)

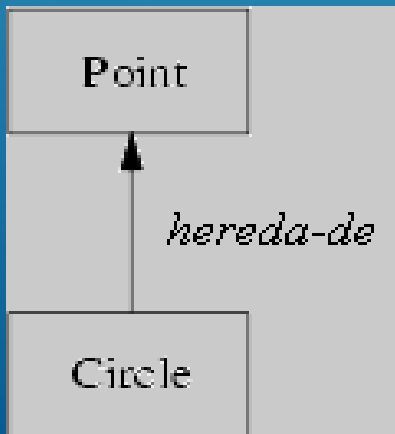
PARADIGMA ORIENTADO A OBJETOS

Def.: Mecanismo q permite q 1 clase A herede prop. de 1 clase B. Se dice q "A hereda de B" cuando Objetos de la clase A tienen acceso a atrib. y mét. de la clase B sin necesidad de redefinirlos.

Def. (Superclase/Subclase): Si la clase A hereda de la clase B, entonces B es la superclase de A. A es subclase de B. Los objetos de 1 subclase pueden ser usados donde son usados los objetos de la superclase. Esto es xq los objetos de la subclase comparten el mismo comportamiento q objetos de la superclase.

Las superclases también se llaman clases padres y las subclases clases hijas o derivadas. También se puede heredar de 1 subclase, haciendo q esta clase sea la superclase de la nueva subclase. Esto conduce a 1 jerarquía de relac. superclase/subclase. Si se dibuja esta jerarquía, se obtiene 1 gráfica de herencia.

1 esquema consiste en usar flechas para indicar la relación de herencia entre clases u objetos.



Gráfica de herencia sencilla

Según el autor puede variar el sentido de la flecha

HERENCIA (cont.) PARADIGMA ORIENTADO A OBJETOS

Herencia Múltiple

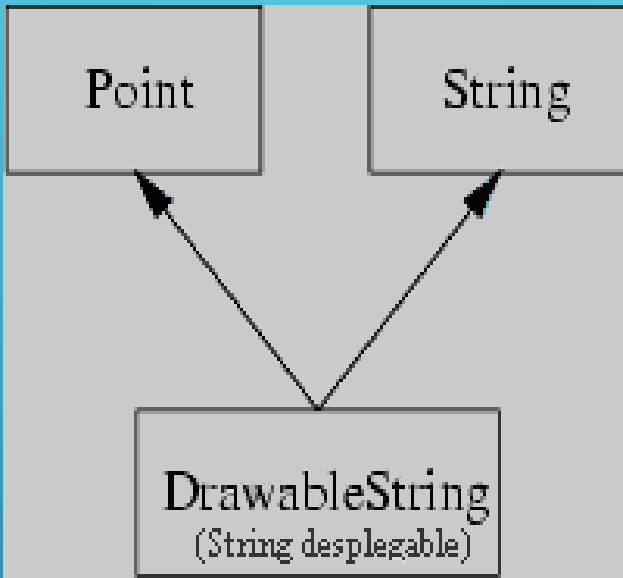


“La herencia múltiple significa q 1 subclase puede tener + de 1 superclase. Esto permite q la subclase herede propiedades de más de una superclase y –mezclar- sus propiedades”.

NO significa q múltiples subclases compartan la misma superclase. Tampoco q1 subclase herede de 1 clase que es a su vez subclase de otra clase.

P/el ej.previo, si se tiene 1 clase String q permite el manejo adecuado de texto. Podría haber, 1 método append p/añadir otro texto. Se quiere usar esta clase p/añadir texto a objetos q se dibujen; y usar rutinas ya existentes tales como move() p/mover el texto donde se necesite.

Es lógico permitir q 1 texto p/dibujarse tenga 1 pto.q defina su localización dentro del área de dibujo. Por lo q se deriva 1 nueva clase DrawableString q hereda propiedades de Point y de String



class DrawableString inherits from Point, String {

attributes: /* Todos heredados de superclases */

methods: /* Todos heredados de superclases */

}

**Se puede usar objetos de la clase DrawableString como ambos:
puntos y strings.**

Debido a que drawablestring es-un(a) point se pueden mover dichos objetos:

DrawableString dstring

...

move(dstring, 10)

...

Desde el momento q son string, se puede añadir otro texto:

dstring.append("La flores color azul ...")

PARADIGMA ORIENTADO A OBJETOS

HERENCIA (cont.) Herencia Múltiple

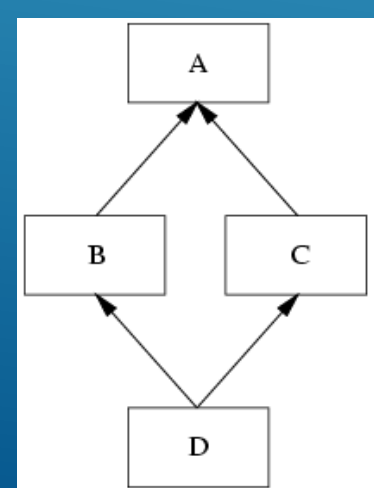
Def.Her.Múlt.: Si la clase A hereda de + de 1 clase, p.ej. A hereda de B1, B2, ..., Bn, se habla de herencia múltiple. Esto puede presentar conflictos de nomenclatura en A si al menos dos de sus superclases definen propiedades (atrib.o métodos) con el mismo nombre.

x ej.si la clase String define un método setX() que pone el string en 1secuencia de "X" caracteres. ¿Q se heredado x DrawableString? ¿La versión de Point, de String o ninguna de las 2?

Estos conflictos se pueden resolver por:

- **El orden en el cuál las superclases son provistas**, definen que propiedad será accesible por el nombre causante del conflicto. Los otros quedarán "escondidos".
- **Las subclases deben resolver el conflicto proveyendo 1prop.con el nombre y definiendo como usar los de sus superclases.**

La 1ra.soluc.no es conveniente ya q presentan consecuencias implíc.dependiendo del orden. En el 2do.caso, las subclases deben redefinir explícitamente las prop.involucradas en conflictos.



Un tipo especial de conflicto de nomenclatura se presenta si 1clase D hereda en forma múltiple de las superclases B y C q a su vez derivan de 1superclase A.

Algunos leng.de progr.resuelven esta gráfica de herencia especial derivando D con

- las prop.de A más
- las prop.de B y C sin las prop. q han heredado de A.

O NO permiten la Herencia Múltiple

PARADIGMA ORIENTADO A OBJETOS

BENEFICIOS QUE SE OBTIENEN DEL DESARROLLO CON OOP

REUTILIZACION DE CODIGO

Los costos del HW decrecen, apareciendo nuevas áreas de aplicación (procesamiento de imágenes y sonido, bases de datos, etc), pero los costos de producción de SW siguen aumentando (mantenimiento y modificación de sist.complejos).

Estos problemas no han sido solucionados en forma completa, pero como los objetos son portables (teóricamente) mientras q la herencia permite la reusabilidad del código OO, es + sencillo modificar código existente xq los objetos no interactúan excepto a través de mensajes; en consecuencia un cambio en la codificación de un objeto no afectará la operación con otro objeto siempre q los métodos respectivos permanezcan intactos.

“La introducción de tecnología de objetos como una herramienta conceptual p/analizar, diseñar e implementar aplicaciones permite obtener aplicaciones más modificables, fácilmente extendibles y a partir de componentes reusables”.

Esta reusabilidad del código disminuye el tiempo que se utiliza en el desarrollo y hace que el desarrollo del software sea más intuitivo porque la gente piensa naturalmente en términos de objetos más que en términos de algoritmos de software.



PARADIGMA ORIENTADO A OBJETOS

Problemas derivados de la utilización de OOP

- a) **Curvas de aprendizaje largas:** Un sist.orient.a objetos ve al mundo en 1 forma única. Involucra la conceptualización de todos los elementos de un progr., desde subsist. a datos, en forma de objetos.
- b) **Dependencia del lenguaje:** A pesar de la portabilidad conceptual de los objetos en un Sist.OO, en la práctica existen muchas dependencias (C++ soporta herencia múltiple mientras que SmallTalk no).
- c) **Determinación de las clases:** 1 clase es un molde q se utiliza p/crear nuevos objetos. Es importante crear el conj.de clases adecuado p/un proyecto. Pero la def.de las clases es +un arte que 1ciencia.
- d) **Performance:** En un sist.donde todo es 1objeto y toda interacción es a través de mensajes, el tráfico de mjes afecta la performance. 1 diseño de 1 aplicación OO q no tiene en cta. la performance no será viable comercialmente

UTOPIA: “ Debería existir una metodología fácil de aprender e independiente del lenguaje, y fácil de reestructurar que no drene la performance del sistema.”

BIBLIOGRAFIA

► Páginas web consultadas:

<https://www.desy.de/gna/html/cc/Tutorial/Spanish/tutorial.html>

Documento original:

<https://www.dropbox.com/s/5we40yhv45yv9s1/Programaci%C3%B3n%20Orientada%20a%20Objetos.doc?dl=0>